

LAB

Topic 9: Transactions & Isolation Levels



Lecturer: Lê Thị Nhàn, PhD.

Lab instructor: Nguyễn Ngọc Toàn, MSc.

Nguyễn Ngọc Minh Châu, MSc.

1. Transactions in SQL

ACID - 4 crucial properties of a transaction:

- **Atomicity**: A transaction, composed of various operations, must satisfy the condition that either **all** operations succeed, or **none** of them do.
- **Consistency**: A transaction will maintain an invariant state of data, meaning the data is always **preserved**. If an error occurs, it reverts to the previous state.
- **Isolation**: A transaction that is currently executing and has not yet been committed must ensure its **independence** from other transactions.
- **Durability** : When a transaction is successfully committed, the data will be accurately saved and remain **persistent**.

a) Commit/Rollback

Example: Transfer Student 'ST005' from NULL department to 'CS' department.

BEGIN TRANSACTION;

PRINT 'Starting Transaction for ST005 transfer...';

-- Update the student's department

UPDATE Student

SET department_id = 'CS'

WHERE student_id = 'ST005';

COMMIT;

COMMIT applies changes into database

BEGIN TRANSACTION;

PRINT 'Starting Transaction for ST005 transfer...';

-- Update the student's department

UPDATE Student

SET department_id = 'CS'

WHERE student_id = 'ST005';

ROLLBACK ;

ROLLBACK discards changes applied to database

b) Try/Catch

The **commit/rollback** are usually used with try/catch

```

BEGIN TRANSACTION;
BEGIN TRY
    PRINT 'Starting Transaction for ST005 transfer...';

    -- Update the student's department
    UPDATE Student
    SET department_id = 'CS'
    WHERE student_id = 'ST005';

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    THROW 50000, 'Cannot transfer', 1
END CATCH;

```

2. Isolation Levels

There are 4 standard isolation levels: **READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ** and **SERIALIZABLE**. SQL Server also has the 5th isolation level **SNAPSHOT** which is not mentioned in this lab.

Let's try each isolation level with the transfer procedure. Note: The default isolation level is **READ COMMITTED**, thus dirty reading cannot happen.

Transaction isolation level	Dirty reads	Nonrepeatable reads	Phantoms
Read uncommitted	x	x	x
Read committed (default)	-	x	x
Repeatable read	-	-	x
Serializable	-	-	-

a) READ UNCOMMITTED

Create department_transfer procedure then update the department ID of a student who has studentID.

```
CREATE PROCEDURE department_transfer(@studentID varchar(9), @todepartmentId
varchar(5))
AS
BEGIN TRANSACTION;
    PRINT 'Transaction started.';
    -- Update the student's department but DO NOT ROLLBACK yet
    UPDATE Student
    SET department_id = @todepartmentId
    WHERE student_id = @studentID;

    PRINT 'Waiting for 5 seconds.';
    WAITFOR DELAY '00:00:05'; -- Simulate a delay for another session to read
    PRINT 'Delay finished.';

    ROLLBACK;
    PRINT 'Transaction rolled back successfully.';
```

Try to execute the department_transfer first with valid parameters. It would take at least 5 seconds to complete:

```
exec department_transfer 'ST005', 'AI';
```

The **WAITFOR DELAY** simulates the real scenario when the database may be busy and execute the procedure too long thus making the concurrent problem easy to produce.

While the above procedure is running, we open a new window (session) to read the updated data.

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT student_id, student_name, department_id
FROM Student
WHERE student_id = 'ST005';
COMMIT TRANSACTION;
```

Remember the student ST005's departmentID and try to run again after the transfer procedure is completed. You would see the difference between 2 times getting the departmentID because the transaction has READ UNCOMMITTED isolation level, it's reading the dirty value of departmentID because the procedure didn't commit/rollback. In this situation, we call this phenomenon "**Dirty Read**".

b) READ COMMITTED

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT student_id, student_name, department_id
FROM Student
WHERE student_id = 'ST005';
COMMIT TRANSACTION;
```

Try again with the **READ COMMITTED** (or higher level). The transaction should wait for the procedure completed then return the latest balance account.

READ COMMITTED ensured all the data is committed before read but not ensure it should read the same value as previous statement in the transaction because other transactions may commit new value.

Now try the following scenario:

1. Execute the department_transfer_commit:

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT student_id, student_name, department_id
FROM Student
WHERE student_id = 'ST005';
WAITFOR DELAY '00:00:05' - - Wait for another transaction executed
SELECT student_id, student_name, department_id
FROM Student
WHERE student_id = 'ST005';
COMMIT;
```

2. Run the following transaction (in separate window):

```
CREATE PROCEDURE department_transfer_commit(@studentID varchar(9),
@todepartmentId varchar(5))
AS
BEGIN TRANSACTION;
```

```
-- Update the student's department
UPDATE Student
SET department_id = @todepartmentId
WHERE student_id = @studentID;
COMMIT;

GO

exec department_transfer_commit 'ST005', 'AI';
```

3. Check the results in the window which runs step 1. We should observe 2 different outcomes from 2 reads in the transaction in step 1. This happens because the transaction in step 2 alters the department_ID of that student after the transaction 1's first read and before that transaction's second read. This phenomenon is known as a **non-repeatable read**.

c) REPEATABLE READ

Now we changed the isolation level in the step 1 in previous scenario into

```
BEGIN TRANSACTION
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT student_id, student_name, department_id
FROM Student
WHERE student_id = 'ST005';
WAITFOR DELAY '00:00:05' - - Wait for another transaction execute
SELECT student_id, student_name, department_id
FROM Student
WHERE student_id = 'ST005';
```

and then rerun the scenario, The second SELECT should output the same result as the first SELECT in the above transaction.

REPEATABLE READ ensures the statement with the same criteria output the same value because another process is blocked from **modifying (or removing)** the data which matched the criteria. They should wait for the **REPEATABLE READ** transaction to complete to update data.

The repeatable read **doesn't guarantee** the new data that also match the criteria not appear in the second call (**insert new row**).

Example: Consider this scenario:

1. Run this transaction in window 1:

```
-- Phantom reads
BEGIN TRANSACTION
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM Student WHERE department_id = 'CS';
WAITFOR DELAY '00:00:10'
SELECT COUNT(*) FROM Student WHERE department_id = 'CS';
COMMIT;
```

2. Run this transaction in window 2 right after:

```
BEGIN TRANSACTION;
    -- Add new student
INSERT INTO Student VALUES ('ST021', 'New Student', 'M', '2004/01/01', '23TT3',
'CS');
COMMIT;
```

=> Despite the isolation level being repeatable read mode, we still observed 2 different results in window 1. To prevent this to happen, we need a higher isolation level, which is **serializable**.

d) SERIALIZABLE

This isolation level ensures all parallel transactions are executed as they run one by one (in an order). Let's change the above example to this isolation level. The latter would fail due to the row with the same id being locked by the first one. When two processes try to lock the same resource, 1 of them would be chosen to die.

e) SELECT with the locks

Select with an update lock (**UPDLOCK**) will lock the rows which match criteria until the transaction is completed. Other transactions with or without a lock may read those rows or insert new ones that match criteria but do not delete or update.

Select with a **hold lock** will lock other transactions not to insert, update or delete.

```
SELECT * FROM table_name WITH (NOLOCK) WHERE ID = 1 -- similar to READ
UNCOMMITTED
SELECT * FROM table_name WITH (READCOMMITTED) WHERE ID = 1 -- similar to READ
COMMITTED
SELECT * FROM table_name WITH (UPDLOCK) WHERE ID = 1 -- similar to REPEATABLE
READ
SELECT * FROM table_name WITH (HOLDLOCK) WHERE ID = 1 -- similar to
SERIALIZABLE isolation
```